Лабораторная работа №2

Архитектура ЭВМ

Альманасра Рами

Content

Цель работы 5

Задание 6

Теоретическое введение 7

Выполнение лабораторной работы 9

Задание для самостоятельной работы 17

Выводы 21

Список литературы 22

Список иллюстраций

1 Предварительная конфигурация git 9
2 Настройка кодировки 9
3 Создание имени для начальной ветки 9
4 Параметр autocrlf 10
5 Параметр safecrlf 10
6 Генерация SSH-ключа 10
7 Установка утилиты xclip 11
8 Копирование содержимого файла 11
9 Окно SSH and GPG key 11
10 Добавление ключа 12
11 Создание рабочего пространства 12
12 Страница шаблона для репозитория 13
13 Окно создания репозитория 13
14 Перемещение между директориями 14
15 Клонирование репозитория 14
16 Окно с ссылкой для копирования репозитория 14
17 Перемещение между директориями 15
18 Удаление файлов 15
19 Создание каталогов 15
20 Добавление и сохранение изменений на сервере 15
21 Выгрузка изменений на сервер 16
22 Страница репозитория 16

1 Создание файла 17
2 Меню приложений 17
3 Работа с отчетом в текстовом процессоре 18
4 Перемещение между директориями 18
5 Проверка местонахождения файлов 18
6 Копирование файла 18
7 Добавление файла на сервер 18
8 Подкаталоги и файлы в репозитории 19
9 Отправка в центральный репозиторий сохраненных изменений . 19
10 Страница каталога в репозитории 19
11 Страница последних изменений в репозитории 20

Список таблиц

Цель работы

Целью работы является изучить идеологию и применение средств контроля версий. Приобрести практические навыки по работе с системой git.

Задание

1. Настройка GitHub.
2. Базовая настройка Git.
3. Создание SSH-ключа.
4. Создание рабочего пространства и репозитория курса на основе шаблона.
5. Создание репозитория курса на основе шаблона.
6. Настройка каталога курса.
7. Выполнение заданий для самостоятельной работы.

Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных. Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить изменения, сделанные разными участниками, вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позво-

ляет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом привилегированный доступ только одному пользователю, работающему с файлом. Системы контроля версий также могут обеспечивать дополнительные,более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступк которому можно ограничить. В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным. Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд. Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды git с различными опциями. Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией. Работа пользователя со своей веткой начинается с проверки и получения изменений из центрального репозитория (при этом в локальное дерево до начала этой процедуры не должно было вноситься изменений). Затем можно вносить изменения в локальном дереве и/или ветке. После завершения внесения какого-то изменения в файлы и/или каталоги проекта необходимо разместить их в центральном репозитории.

Выполнение лабораторной работы

Открываю виртуальную машину, затем открываю терминал и делаю предварительную конфигурацию git. Ввожу команду git config –global user.name “”, указывая свое имя и команду git config –global user.email “work@mail”, указывая в ней электронную почту владельца, то есть мою (@fig:001)

```
ramialmanasra@vbox:~$ git config --global user.name "<Rami Almanasra>"
ramialmanasra@vbox:~$ git config --global user.email "<ramialmanasra@gmail.com>"
ramialmanasra@vbox:~$
```

Рис. 1: Предварительная конфигурация git

Настраиваю utf-8 в выводе сообщений git для корректного отображения символов (@fig:002)

```
ramialmanasra@vbox:~$ git config --global core.quotepath false
ramialmanasra@vbox:~$
```

Рис. 2: Настройка кодировки

Задаю имя «master» для начальной ветки (@fig:003)

```
ramialmanasra@vbox:~$ git config --global init.defaultBranch master
ramialmanasra@vbox:~$
```

Рис. 3: Создание имени для начальной ветки

Задаю параметр autocrlf со значением input и параметр safecrlf со значением warn (@fig:004) (@fig:005)

```
ramialmanasra@vbox:~$ git config --global core.autocrlf input
ramialmanasra@vbox:~$
```

Рис. 4: Параметр autocrlf

```
ramialmanasra@vbox:~$ git config --global core.safecrlf warn
ramialmanasra@vbox:~$
```

Рис. 5: Параметр safecrlf

Для последующей идентификации пользователя на сервере репозиториев необходимо сгенерировать пару ключей (приватный и открытый). Для этого ввожу команду ssh-keygen -C “Имя Фамилия, work@email”, указывая имя владельца и электронную почту владельца (@fig:006). Ключ автоматически сохранится в каталоге ~/.ssh/.

```
ramialmanasra@vbox:~$ ssh-keygen -C "Rami Almanasra <ramialmanasra@gmail.com>"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ramialmanasra/.ssh/id_ed25519):
Created directory '/home/ramialmanasra/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ramialmanasra/.ssh/id_ed25519
Your public key has been saved in /home/ramialmanasra/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:5LZyvYMHT2nJ7JDZOQekqluD7kFAw7yj7qYwPtWqyjs Rami Almanasra <ramialmanasra@gmail.com>
The key's randomart image is:
+--[ED25519 256]--+
| oo              |
| .o.      .      |
|  ..    .o       |
|  o.   o. .      |
| . .o  .S* =     |
|.  o o..=o@ .    |
|+ . +.+ oO.o     |
|+E o.o +. =.     |
|B*=.+.   ...     |
+----[SHA256]-----+
```

Рис. 6: Генерация SSH-ключа

Xclip – утилита, позволяющая скопировать любой текст через терминал. Оказывается, в дистрибутиве Linux Fedora ее сначала надо установить. Устанавливаю xclip с помощью команды dnf install с ключом -y отимени суперпользователя, введя в начале команды sudo (@fig:007)

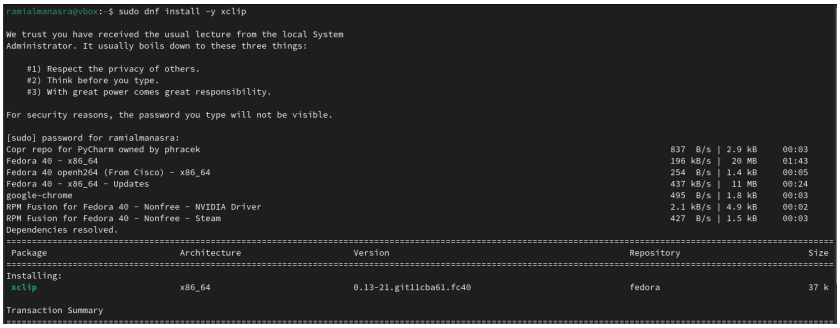

Рис. 7: Установка утилиты xclip

Копирую открытый ключ из директории, в которой он был сохранен, с помощью утилиты xclip (@fig:008)

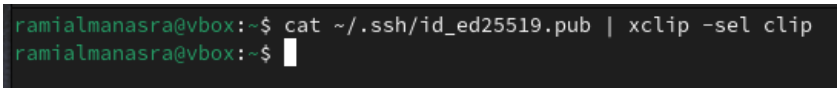

Рис. 8: Копирование содержимого файла

Открываю браузер, захожу на сайт GitHub. Открываю свой профиль и выбираю страницу «SSH and GPG keys». Нажимаю кнопку «New SSH key» (@fig:009)

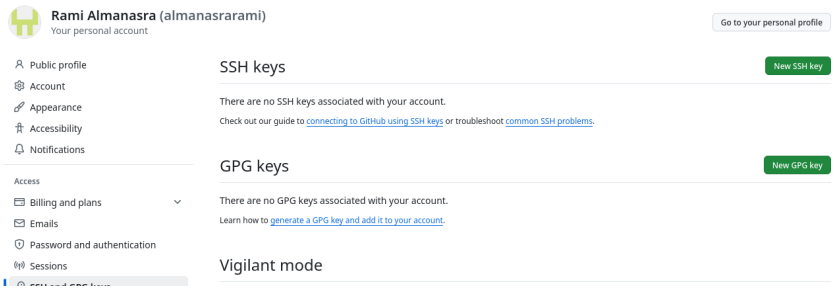

Рис. 9: Окно SSH and GPG key

Вставляю скопированный ключ в поле «Key». В поле Title указываю имя для ключа. Нажимаю «Add SSH-key», чтобы завершить добавление ключа (@fig:010)

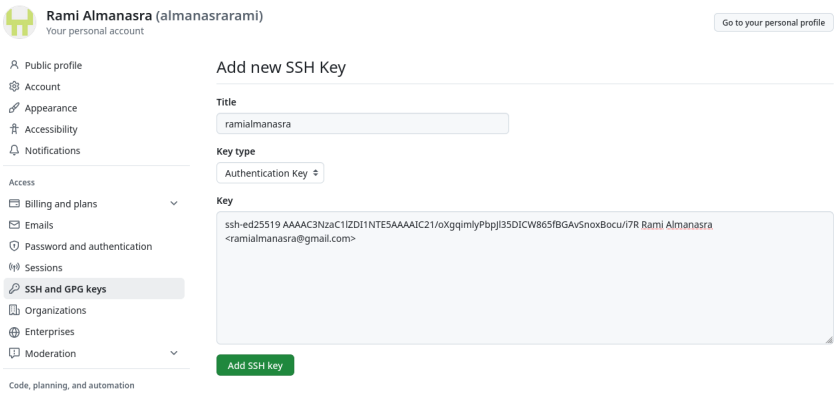

Рис. 10: Добавление ключа

Закрываю браузер, открываю терминал. Создаю директорию, рабочее пространство, с помощью утилиты mkdir, блягодаря ключу -p создаю все директории после домашней ~/work/study/2024-2025/“ Операционные системы” рекурсивно. Далее проверяю с помощью ls, действительно ли были созданы необходимые мне каталоги (@fig:011)

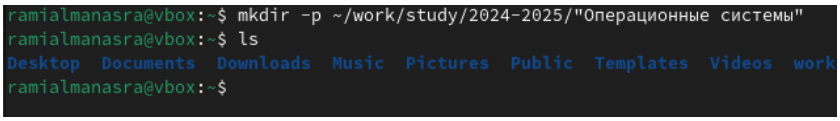

Рис. 11: Создание рабочего пространства

В браузере перехожу на страницу репозитория с шаблоном курса по адресу https://github.com/yamadharma/course-directory-student-template. Далее выбираю «Use this template», чтобы использовать этот шаблон для своего репозитория (@fig:012)

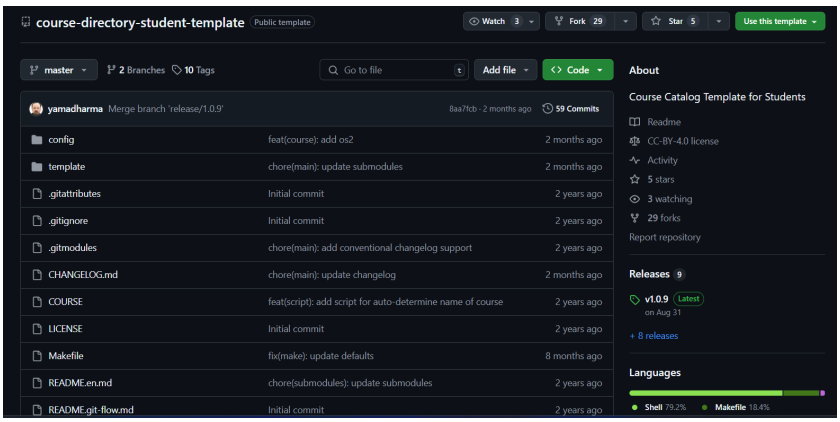

Рис. 12: Страница шаблона для репозитория

В открывшемся окне задаю имя репозитория (Repository name): study_2024–2025_os intro и создаю репозиторий, нажимаю на кнопку «Create repository from template» (@fig:013)

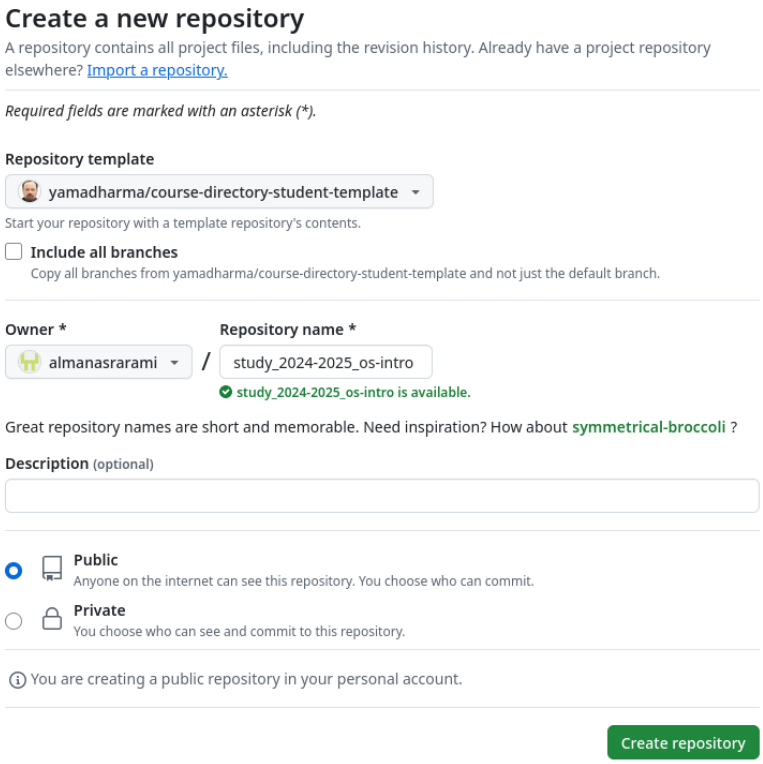

Рис. 13: Окно создания репозитория

Через терминал перехожу в созданный каталог курса с помощью утилиты cd (@fig:014) и Клонирую созданный репозиторий с помощью команды git clone –recursive git@github.com:/study_2024–2025_os-intro.git os-intro (@fig:015)

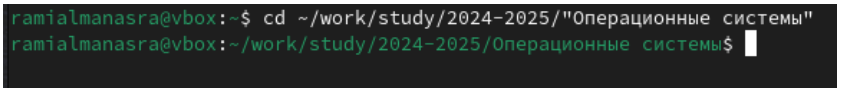

Рис. 14: Перемещение между директориями

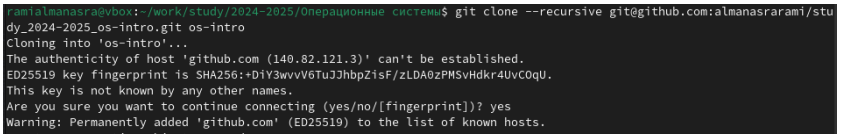

Рис. 15: Клонирование репозитория

Копирую ссылку для клонирования на странице созданного репозитория, сначала перейдя в окно «code», далее выбрав в окне вкладку «SSH» (@fig:016)

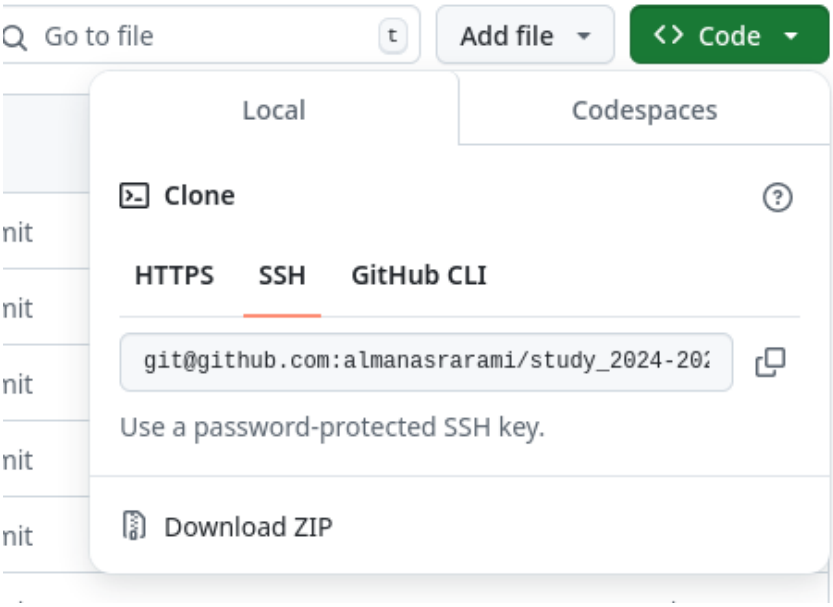

Рис. 16: Окно с ссылкой для копирования репозитория

Перехожу в каталог os-intro с помощью утилиты cd (@fig:017) и Удаляю лишние

файлы с помощью утилиты rm (@fig:018)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы$ cd ~/work/study/2024-2025/"Операционные системы"/os-in
tro
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$
```

Рис. 17: Перемещение между директориями

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ rm package.json
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$
```

Рис. 18: Удаление файлов

Создаю необходимые каталоги (@fig:019)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ echo os-intro > COURSE
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ make prepare
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$
```

Рис. 19: Создание каталогов

Отправляю созданные каталоги с локального репозитория на сервер: добавляю все созданные каталоги с помощью git add, комментирую и сохраняю изменения на сервере как добавление курса с помощью git commit (@fig:020)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ git add .
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ git commit -am 'feat(main): make course struc
ture'
[master ce4312f] feat(main): make course structure
 405 files changed, 98413 insertions(+), 14 deletions(-)
 create mode 100644 labs/README.md
 create mode 100644 labs/README.ru.md
 create mode 100644 labs/lab01/presentation/.projectile
 create mode 100644 labs/lab01/presentation/.texlabroot
 create mode 100644 labs/lab01/presentation/Makefile
 create mode 100644 labs/lab01/presentation/image/kulyabov.jpg
 create mode 100644 labs/lab01/presentation/presentation.md
 create mode 100644 labs/lab01/report/Makefile
 create mode 100644 labs/lab01/report/bib/cite.bib
 create mode 100644 labs/lab01/report/image/placeimg_800_600_tech.jpg
 create mode 100644 labs/lab01/report/pandoc/csl/gost-r-7-0-5-2008-numeric.csl
 create mode 100755 labs/lab01/report/pandoc/filters/pandoc_eqnos.py
 create mode 100755 labs/lab01/report/pandoc/filters/pandoc_fignos.py
 create mode 100755 labs/lab01/report/pandoc/filters/pandoc_secnos.py
 create mode 100755 labs/lab01/report/pandoc/filters/pandoc_tablenos.py
 create mode 100644 labs/lab01/report/pandoc/filters/pandocxnos/__init__.py
 create mode 100644 labs/lab01/report/pandoc/filters/pandocxnos/core.py
 create mode 100644 labs/lab01/report/pandoc/filters/pandocxnos/main.py
 create mode 100644 labs/lab01/report/pandoc/filters/pandocxnos/pandocattributes.py
 create mode 100644 labs/lab01/report/report.md
 create mode 100644 labs/lab02/presentation/.projectile
 create mode 100644 labs/lab02/presentation/.texlabroot
 create mode 100644 labs/lab02/presentation/Makefile
 create mode 100644 labs/lab02/presentation/image/kulyabov.jpg
 create mode 100644 labs/lab02/presentation/presentation.md
```

Рис. 20: Добавление и сохранение изменений на сервере

Отправляю все на сервер с помощью push (@fig:021)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro$ git push
Enumerating objects: 40, done.
Counting objects: 100% (40/40), done.
Delta compression using up to 4 threads
Compressing objects: 100% (30/30), done.
Writing objects: 100% (38/38), 341.67 KiB | 540.00 KiB/s, done.
Total 38 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To github.com:almanasrarami/study_2024-2025_os-intro.git
   bc519ae..ce4312f  master -> master
```

Рис. 21: Выгрузка изменений на сервер

Проверяю правильность выполнения работы сначала на самом сайте GitHub (@fig:022)

almanasrarami feat(main): make course structure ce4312f · 1 minute ago

Name	Last commit message	Last commit date
..		
lab01	feat(main): make course structure	1 minute ago
lab02	feat(main): make course structure	1 minute ago
lab03	feat(main): make course structure	1 minute ago
lab04	feat(main): make course structure	1 minute ago
lab05	feat(main): make course structure	1 minute ago
lab06	feat(main): make course structure	1 minute ago
lab07	feat(main): make course structure	1 minute ago
lab08	feat(main): make course structure	1 minute ago
lab09	feat(main): make course structure	1 minute ago
lab10	feat(main): make course structure	1 minute ago
lab11	feat(main): make course structure	1 minute ago
lab12	feat(main): make course structure	1 minute ago

Рис. 22: Страница репозитория

Задание для самостоятельной работы

1. Перехожу в директорию labs/lab02/report с помощью утилиты cd. Создаю в каталоге файл для отчета по второй лабораторной работе с помощью утилиты touch (@fig:023)

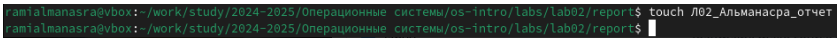

Рис. 1: Создание файла

Оформить отчет я смогу в текстовом процессоре LibreOffice Writer, найдя его в меню приложений (@fig:024)

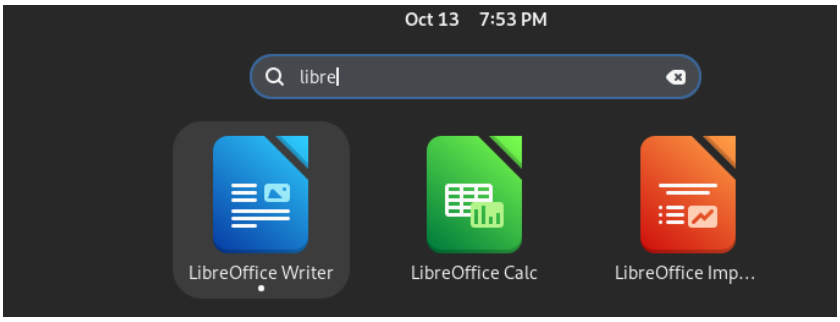

Рис. 2: Меню приложений

После открытия текстового процессора открываю в нем созданный файл и могу начать в нем работу над отчетом (@fig:025)

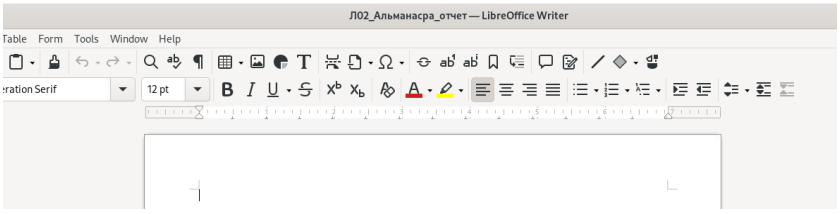

Рис. 3: Работа с отчетом в текстовом процессоре

2. Перехожу из подкаталога lab02/report в подкаталог lab01/report с помощью утилиты cd (@fig:026) и Проверяю местонахождение файла с отчетом по первой лабораторной работой. Он должен быть в подкаталоге домашней директории «Downloads», для проверки использую команду ls (@fig:027)

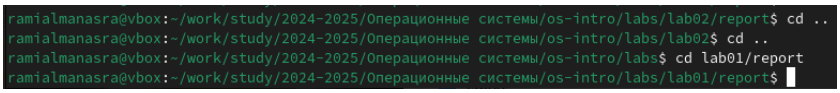

Рис. 4: Перемещение между директориями

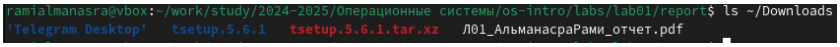

Рис. 5: Проверка местонахождения файлов

Копирую первую лабораторную с помощью утилиты cp и проверяю правильность выполнения команды cp с помощью ls (@fig:028)

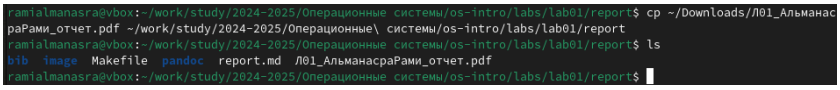

Рис. 6: Копирование файла

Добавляю с помощью команды git add в коммит созданные файлы: Л02_Альманасра_отчет (@fig:029)

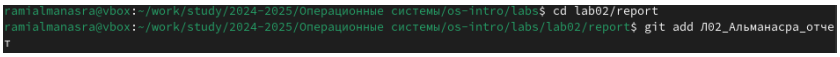

Рис. 7: Добавление файла на сервер

Сохраняю изменения с помощью git commit (@fig:030)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab02/report$ git commit -m "add existing
 file"
[master a1ba447] add existing file
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 labs/lab02/report/Л02_Альманасра_отчет
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab02/report$
```

Рис. 8: Подкаталоги и файлы в репозитории

Отправляю в центральный репозиторий сохраненные изменения командой git push -f origin master (@fig:031)

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab02/report$ git push -f origin master
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 508 bytes | 508.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:almanasrarami/study_2024-2025_os-intro.git
   ce4312f..a1ba447  master -> master
```

Рис. 9: Отправка в центральный репозиторий сохраненных изменений

Проверяю на сайте GitHub правильность выполнения заданий. Вижу, что пояснение к совершенным действиям отображается (@fig:032)

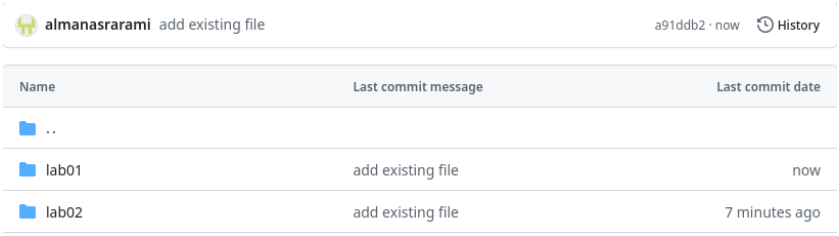

Рис. 10: Страница каталога в репозитории

При просмотре изменений так же вижу, что были добавлены файлы с отчетами по лабораторным работам (@fig:033)

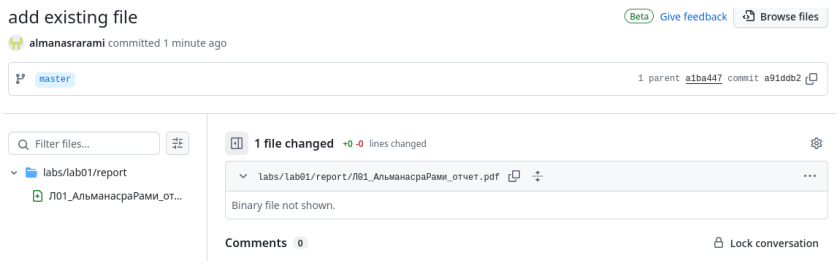

Рис. 11: Страница последних изменений в репозитории

Выводы

При выполнении данной лабораторной работы я изучила идеологию и применение средств контроля версий, а также приобрела практические навыки по работе с системой git.

Список литературы

1. Course on TUIS
2. Laboratory work No. 2